R3.01 - DÉVELOPPEMENT WEB

Sujet projet

BUT Informatique, 2e année | Année universitaire 2025/2026 | Desbenoit B.

Introduction

En vue de valider les compétences acquises dans le module R3.01 - Développement web, vous devez réaliser le développement d'une application client-riche. L'objectif de cette application consiste avant toute chose à montrer que vous êtes capable de mettre en œuvre les techniques vues tout au long du cours.

Le choix de l'application vous revient dès lors que vous respectez les prérequis ci-dessous :

- Programmation orientée objet
- Application exportée via un module empaqueté par webpack
- Compatibilité avec au moins les 3 derniers navigateurs
- Toutes les dépendances sans exception devront être gérées par webpack (ex : pas de CDN, pas d'inclusion de script manuelle dans l'HTML, une image = une dépendance, etc.).
- Le contenu de l'application doit reposer sur au moins deux requêtes XHR (ou Fetch) dépendantes l'une de l'autre. Par exemple pour une application météo, il faut d'abord récupérer l'identifiant d'une ville pour en récupérer ensuite la météo. À noter que les deux requêtes peuvent aussi être réalisées sur deux API différentes. Par exemple une pour récupérer une adresse, et une autre qui l'utilise pour afficher les stations de vélos disponibles les plus proches.

Le **choix** de votre **application** devra être **validé par l'enseignant** avant la fin de la **séance du 09/10/2026**.

Vous trouverez ci-dessous quelques liens vers des API CORS pour vous inspirer :

- Wikipedia : https://www.mediawiki.org/wiki/API:Main_page (https://www.mediawiki.org/wiki/API:Main_page)
- Navitia : https://doc.navitia.io/#getting-started (https://doc.navitia.io/#getting-started)
- API de Laposte : https://datanova.laposte.fr/datasets (https://datanova.laposte.fr/datasets) (Localisation des bureaux de postes, des boites aux lettres de rue, etc.)
- API de recherche d'adresses, à combiner avec une autre pour être vraiment pertinente : https://adresse.data.gouv.fr/api (https://adresse.data.gouv.fr/api)
- Plein d'autres API ouvertes : https://data.opendatasoft.com/explore/ (https://data.opendatasoft.com/explore/), ou encore : https://blog.shevarezo.fr/post/2015/03/17/10-apis-gratuites-utiles (https://blog.shevarezo.fr/post/2015/03/17/10-apis-gratuites-utiles)

Vous êtes fortement invité à ne pas vous limiter au minimum demandé. Plus vous irez loin, plus vous apprendrez, et meilleure sera la note.

Planning

Le projet final est à rendre pour le **23/10/2026 avant 23h59**, délai de rigueur.

Versionner son projet avec git

Git est un outil de gestion de versions qui fonctionne **entièrement en local**, sur votre machine. Vous n'avez besoin ni de GitHub, ni de GitLab, ni d'aucun serveur. Toutes les versions de votre projet sont stockées dans un dossier caché `.git` à la racine de votre projet.

Pour ce projet, l'utilisation de git est **obligatoire**. L'historique de vos commits fait partie du rendu.

Initialiser le dépôt (une seule fois par projet)

Placez-vous à la racine de votre projet, puis :

```
git init
```

Indiquez ensuite votre identité, qui apparaîtra dans chacun de vos commits :

```
git config user.name "Prénom Nom"
git config user.email "prenom.nom@etu.univ-amu.fr"
```

Sans l'option `--global`, cette configuration ne s'applique qu'à ce projet (elle est enregistrée dans `.git/config`). C'est la solution à privilégier sur les machines partagées de l'IUT. Sur votre machine personnelle, vous pouvez ajouter `--global` pour ne la saisir qu'une seule fois pour tous vos projets.

Créez enfin un fichier `.gitignore` à la racine pour exclure les fichiers qui ne doivent pas être versionnés :

```
node_modules/
dist/
```

Le cycle de travail

À chaque étape significative (une classe terminée, une requête qui fonctionne, un bug corrigé) :

```
git status                  # voir les fichiers modifiés
git add .                   # préparer toutes les modifications
git commit -m "Ajout de la classe Ville et de la requête de recherche"
```

Pour consulter l'historique :

```
git log --oneline
```

Pour voir ce que vous avez modifié depuis le dernier commit :

```
git diff
```

Pour annuler les modifications d'un fichier et revenir à la dernière version commitée :

```
git restore chemin/du/fichier.js
```

> 💡 VS Code intègre git nativement (onglet *Contrôle de code source*). Vous pouvez faire tout ce qui précède sans passer par le terminal.

Bonnes pratiques

- **Commitez souvent**, au fil de l'eau, et pas uniquement à la fin. Plusieurs commits par séance de travail est un bon rythme.
- **Rédigez des messages explicites** qui décrivent ce que fait le commit : « Gestion des erreurs de la requête météo » plutôt que « modif » ou « ça marche ».
- **Un commit = une modification cohérente.** Évitez de mélanger plusieurs fonctionnalités dans un même commit.
- Si vous utilisez malgré tout un dépôt distant (GitHub, GitLab…), il doit être **privé**. Un dépôt public rend votre code accessible aux autres et vous expose aux règles sur le plagiat.

Utilisation de l'IA générative

L'usage d'outils d'IA (ChatGPT, Copilot, Claude, etc.) n'est pas interdit, mais il est encadré. L'objectif de ce projet est de montrer que **vous** savez concevoir et développer une application. Une application que vous ne savez pas expliquer ne démontre aucune compétence.

Usages autorisés

- Vous faire expliquer une notion, un message d'erreur ou la documentation d'une API.
- Demander une relecture ou des pistes d'amélioration sur du code que vous avez écrit.
- Générer de petits fragments ponctuels (une expression régulière, une option de configuration webpack…) que vous comprenez et adaptez.

Obligations

- **Historique git** : le projet doit être versionné avec git (voir la section précédente). Les commits doivent être réguliers et refléter l'avancement réel du travail. Un historique réduit à quelques commits massifs sera pénalisé.
- **Déclaration d'usage** : le readme doit contenir une section « Usage de l'IA » indiquant les outils utilisés, pour quoi faire, et les parties du code concernées. Si vous n'avez pas utilisé d'IA, indiquez-le. Une déclaration honnête n'est pas pénalisée. Un usage non déclaré ou non autorisé le sera.
- **Choix de conception** : le readme doit contenir une section « Choix de conception » répondant, avec vos propres mots et en lien direct avec votre code, aux questions suivantes :
  - Pourquoi avez-vous découpé votre application avec ces classes ?
  - Comment l'enchaînement de vos deux requêtes dépendantes est-il géré ?
  - Quelle difficulté principale avez-vous rencontrée, et comment l'avez-vous résolue ?

Modalité de rendu

Le rendu de votre projet se fera par un dépôt sur AMETICE. Tout retard sera sanctionné à hauteur de 1 point par heure, la date et l'heure de la validation du dépôt faisant foi. Attention à bien lire et respecter les consignes qui vous seront fournies pour l'upload de votre projet.

Contenu de l'archive

Votre archive devra contenir :

- Votre projet, **sans** le dossier `node_modules`.
- Le dossier `.git` de votre projet. Comme il s'agit d'un dossier caché, vérifiez qu'il figure bien dans votre archive avant le dépôt.
- Un fichier readme avec les informations suivantes :
  - Votre nom, prénom, mail universitaire
  - Une description exhaustive de votre projet
  - Les commandes nécessaires pour l'installer et l'exécuter
  - Une section « Usage de l'IA »
  - Une section « Choix de conception »

Critères d'évaluation

Vous serez évalué sur les points suivants :

- Éléments de conception (un diagramme de classes peut être fourni avec le projet)
- Qualité de la mise en œuvre
- Respect des contraintes techniques
- Justification des choix et traçabilité du travail (historique git, readme)

Ce travail donnera une note sur 20. Attention, comme dit en cours, ce travail est individuel. Il ne vous est pas permis de partager votre code entre vous, ni de réutiliser du code trouvé par ailleurs qui répondrait au travail demandé. L'usage de l'IA générative est encadré par la section dédiée ci-dessus. Votre code sera passé dans un analyseur permettant de détecter les plagiats.

©DESBENOIT Brett - IUT d'Aix-Marseille